Agent Guardrail Auditor

A safety framework for auditing autonomous LLM agent actions

Prepared by Anjali Tapare

https://github.com/tapareanjali391-tech/agent-guardrail-auditor

Motivation

As LLM agents gain the ability to take autonomous real-world actions — browsing websites, submitting forms, making purchases, and sending communications — the cost of an unchecked mistake scales with every step the agent takes without oversight. A single runaway loop can spam applications, spend money, or leak credentials before a human notices. This project wraps an agent's action loop with programmatic guardrails: every proposed action is evaluated against authored safety policies before execution, and untrusted context is scanned for prompt-injection attempts before it reaches the agent.

Architecture

```
Incoming Context (web pages, tool outputs, documents)
                         |
                         v
             +----------------------+
             | PromptInjectionDet.  |---> quarantine suspicious content
             +----------+-----------+
                        | clean content only
                        v
                  +-----------+
                  |   Agent   |
                  +-----+-----+
                        | proposes action
                        v
             +---------------------+
             |   ActionAuditor     |
             |   Policy Check      |
             +----------+----------+
                        v
         ALLOW  |  BLOCK  |  ESCALATE_TO_HUMAN
                        |
                        v
                 audit_log.json
```

Demo Results

Scenario	Verdict
Read job posting	ALLOW
Fabricated experience in application	BLOCK
Honest application submission	ESCALATE_TO_HUMAN
4th submission within an hour (rate limit)	ESCALATE_TO_HUMAN
Premium purchase attempt	BLOCK
Injected prompt in scraped HTML	QUARANTINED

Relevance to AI Safety Research

This project is a practical engineering exploration of problems central to AI safety research: **scalable oversight** (automating routine policy checks while routing high-risk decisions to humans), **corrigibility** (explicit BLOCK and

ESCALATE verdicts create intervention points before irreversible actions), and **defense against prompt injection** (quarantining hijack attempts in untrusted retrieved content). It demonstrates how lightweight guardrails can be layered onto agentic systems today, while surfacing the gaps that production deployments must address.

Limitations & Future Work

- Rule-based pattern matching rather than learned classification
- Policies must be manually authored and may miss novel attack patterns
- Prompt-injection detection relies on regex heuristics, evadable by adversaries
- Production use would require adversarial red-teaming and likely a fine-tuned classifier for injection detection